Two Pointers — Pattern Recognition & Universal Notes

These notes are meant for quick revision before solving any two pointer problem. They focus on recognizing when two pointers apply, the common movement strategies, and the templates used across many problems.

---

1. How to Recognize a Two Pointer Problem

Typical signals:

- "Find pair with target sum"
- "Remove duplicates"
- "Sorted array"
- "Subarray / substring conditions"
- "Move elements in-place"

Two pointers are usually used when:

• We need to compare elements from different positions • We want O(n) instead of $O(n^2)$ • The array is sorted or order matters

Typical examples:

Problem	Idea
Two Sum (sorted)	opposite pointers
Container With Most Water	shrinking window
Remove Duplicates	slow-fast pointer
Move Zeroes	write pointer
3Sum	fix one + two pointers

---

2. Core Idea

Two pointers means maintaining **two indices that move through the array based on a rule**.

Instead of nested loops:

```
for i
   for j
```

We simulate the same logic with **controlled pointer movement**.

Goal:

Reduce complexity from

```
O(n²) → O(n)
```

---

3. Opposite Direction Pointers

Most common when array is **sorted**.

Pointers start at both ends.

```
left  -> start
right -> end
```

Template:

```
int left = 0;
int right = arr.size() - 1;

while (left < right) {

    int sum = arr[left] + arr[right];

    if (sum == target)
        return true;

    if (sum < target)
        left++;
    else
        right--;
}
```

Used in:

- Two Sum II
- Container With Most Water
- Pair sum problems

---

4. Same Direction Pointers (Fast / Slow)

Both pointers move from left to right.

Usually used for **in-place modifications**.

Example: Remove duplicates.

Template:

```
int slow = 0;

for (int fast = 1; fast < n; fast++) {

    if (arr[fast] != arr[slow]) {
        slow++;
        arr[slow] = arr[fast];
    }
}

return slow + 1;
```

Idea:

```
fast → scanning
slow → writing position
```

---

5. Sliding Window (Dynamic Two Pointers)

Two pointers represent a **window**.

```
[left .... right]
```

Right pointer expands the window.

Left pointer shrinks the window.

Template:

```
int left = 0;

for (int right = 0; right < n; right++) {

    // expand window

    while (window invalid) {
        left++;
    }

    // update answer
}
```

Used in:

- Longest substring problems
- Minimum window substring
- Subarray with sum k

---

6. Fix One + Two Pointer Pattern

Common in problems like **3Sum and 4Sum**.

Steps:

1. Fix one element
2. Solve remaining with two pointers

Template:

```
for (int i = 0; i < n; i++) {

    int left = i + 1;
    int right = n - 1;

    while (left < right) {
```

```
        int sum = arr[i] + arr[left] + arr[right];

        if (sum == target) {
            // found
            left++;
            right--;
        }
        else if (sum < target)
            left++;
        else
            right--;
    }
}
```

---

7. Deduplication Trick

Important in problems like **3Sum**.

Avoid duplicate answers.

Template:

```
while (left < right && arr[left] == arr[left-1])
    left++;
```

---

8. When Two Pointers Works Best

Two pointer works best when:

• array is sorted • we are comparing pairs • we need in-place modification • we are maintaining a window

---

9. Mental Checklist Before Coding

Ask yourself:

1. Is the array sorted?
2. Am I comparing pairs?
3. Can two indices replace nested loops?

4. Is this a sliding window problem?
5. Which pointer moves when condition changes?

If movement rule is clear, coding becomes easy.

---

10. Pattern Recognition Summary

Most two pointer problems fall into these patterns:

Pattern	Example
Opposite pointers	Two Sum II
Fast / slow pointers	Remove duplicates
Sliding window	Longest substring
Fix + two pointers	3Sum

Recognizing which pattern applies is the key.

---

11. Universal Two Pointer Templates

Opposite pointers

```
while (left < right)
```

Fast / slow pointer

```
for (fast = 0 → n)
```

Sliding window

```
for (right)
   while (invalid)
      left++
```

Fix + two pointer

```
for i
   two pointers
```

These templates solve most problems in this category.